

# Глава 1. Почему появился Context Engineering

От чат-бота и промпта к AI-агенту, контексту и проверке

Учебный материал по первой главе линии Context Engineering. Простым языком: почему одного хорошего запроса стало мало и зачем агенту нужна рабочая среда.

## Что поймёте

почему тема появилась именно сейчас

## Главная схема

Chat -> Prompt -> Agent -> Context -> Verification

## Практика

карта перехода для своего процесса

## Результат

понимание, где заканчивается промпт и начинается контекст

# Содержание

## Первая часть

- Зачем изучать главу
- Мини-словарь
- От чата к агенту
- Почему prompt engineering был полезен
- Почему агенту нужна рабочая среда
- Почему плохой контекст ломает модель

## Вторая часть

- Почему больше контекста не всегда лучше
- Схема перехода
- Пример с отчётом по клиентам
- Практика и ожидаемый результат
- Самопроверка
- 60-секундное объяснение

**Тема:** почему тема Context Engineering (контекстный инжиниринг - проектирование контекста для AI-систем) возникла именно сейчас и почему она связана с переходом от чат-ботов к AI-агентам.

**Практический артефакт главы:** Chat -> Prompt -> Agent -> Context -> Verification

---

## Зачем изучать эту главу

Если коротко: чтобы не перепутать *красивый запрос* с *рабочей системой*.

Когда человек впервые знакомится с AI, ему часто кажется, что главное - научиться правильно писать запросы. Это правда только на первом этапе. Для обычного чата хороший запрос действительно многое решает. Но как только AI начинает не просто отвечать, а *делать работу*, появляется другая задача: нужно собрать для него нормальную рабочую среду.

Именно здесь и появляется Context Engineering.

Эта глава нужна, чтобы вы поняли:

- почему тема возникла не “потому что рынку захотелось новое модное слово”;
  - почему одного prompt engineering (промт-инжиниринга - умения правильно формулировать запрос) стало мало;
  - почему AI-агенту нужен не только запрос, но и контекст, инструменты, память, ограничения и проверка;
  - почему плохой контекст ломает даже сильную модель;
  - почему “больше контекста” не равно “лучше контекст”.
- 

## Что должно стать понятно после главы

После этой главы человек должен уметь сказать простыми словами:

Context Engineering появился потому, что AI-системы перестали быть просто чатами. Теперь они работают как агенты: получают задачи, обращаются к источникам, используют инструменты, сохраняют состояние, действуют в процессе и должны показывать проверяемый результат.

И ещё:

Prompt (запрос) говорит, что сделать. Context (контекст) показывает, в какой реальности это делать. Context Engineering - это сборка этой реальности.

---

## Мини-словарь

Только те термины, без которых дальше будет каша.

- prompt (промт - запрос к модели): что мы ей говорим.
- context (контекст - всё, что модель видит перед следующим шагом): задача, правила, история, источники, память, инструменты, ограничения.

- AI agent (AI-агент - программа с ролью, инструментами и задачей): не просто отвечает, а делает шаги.
  - tool (инструмент): способ действия агента - поиск, файл, база данных, API, браузер.
  - memory (память): устойчивые факты и правила, которые должны пережить один разговор.
  - workflow (рабочий процесс): порядок действий от задачи до результата.
  - verification (проверка): как понять, что результат действительно готов.
- 

## 1. От чата к агенту: что вообще изменилось

Сначала AI использовали почти как умный чат-бот.

Схема была простой:

```
человек -> запрос -> модель -> ответ
```

В этой схеме задача была понятная: написать хороший запрос, задать роль, уточнить формат, попросить пример. Это и есть полезная база prompt engineering.

Но потом системы начали менять форму.

Появились:

- AI-агенты, которые могут делать несколько шагов подряд;
- tool calling (вызов инструментов - когда модель не только отвечает, но и вызывает внешнюю функцию);
- RAG (retrieval-augmented generation - генерация с подгрузкой релевантных данных из базы знаний);
- memory (память - устойчивые факты и предпочтения);
- MCP (Model Context Protocol - протокол подключения инструментов и источников к модели);
- многошаговые сценарии;
- человеческое одобрение для рискованных действий;
- проверки качества результата.

И вот тут “просто хороший запрос” перестал быть достаточным.

Потому что агенту нужно не только понять просьбу. Ему нужно *правильно работать в среде*.

---

## 2. Почему prompt engineering был полезен

Сначала не будем его хоронить. Он полезен и сейчас.

Prompt engineering помог людям научиться:

- говорить с моделью яснее;
- задавать роль;
- просить нужный формат;
- не лепить в один запрос всё подряд;
- получать более предсказуемый ответ.

Для чат-бота это было уже большим улучшением.

Например, вместо:

Расскажи про продажи.

человек писал:

Объясни простыми словами, как улучшить продажи в B2B, дай 5 пунктов и один пример.

Ответ становился лучше.

Но это всё ещё уровень *разговора*.

А у агента уже другой режим:

задача -> контекст -> инструменты -> действие -> проверка -> результат

Если вы в этом режиме оставите только красивый запрос, агент начнёт угадывать. А угадывание - плохая база для работы.

---

### 3. Почему агентам нужен не только запрос, но и рабочая среда

Когда AI-агент делает работу, ему нужно знать не просто “что от него хотят”, а:

- кто он в этой задаче;
- какие данные ему можно использовать;
- какие источники считать главным правдой;
- какие инструменты ему разрешены;
- что нельзя делать;
- когда надо остановиться и спросить человека;
- как выглядит готовый результат;
- как проверить, что он не ошибся.

То есть нужно собрать не отдельную фразу, а *систему условий*.

Простая аналогия:

- prompt - это как устная просьба;
- context - это как папка на столе перед работой;
- Context Engineering - это как собрать правильную папку, а не набросать туда случайные листы.

Если папка пустая - человек/агент будет гадать. Если папка забита мусором - он утонет. Если папка собрана правильно - работа идёт быстрее и чище.

---

### 4. Почему плохой контекст ломает даже сильную модель

Это ключевая мысль главы.

Сильная модель - не волшебник. Она всё ещё ограничена тем, что ей дали в контекст.

Если контекст плохой, модель может:

- взять не тот источник;
- проигнорировать важный документ;
- смешать старую и новую информацию;

- сделать вывод на основе мусора;
- выполнить не ту задачу;
- “уверенно” ответить неправильно.

Это очень важно понимать: ошибка часто не в модели как таковой, а в том, *что в неё загрузили*.

Хорошая формула из практики:

Модель не слабая. Контекст плохой.

Именно поэтому в агентных системах появляется новая работа: не только писать запрос, а *управлять контекстом*.

---

## 5. Почему “больше контекста” не равно “лучше контекст”

Это частая ловушка.

Многие думают так:

“Давайте просто дадим модели побольше информации - и она разберётся.”

Но длинный контекст сам по себе не делает систему умнее.

Проблемы длинного контекста:

- важная информация теряется в середине;
- модель начинает путать приоритеты;
- лишние данные отвлекают внимание;
- в контекст попадают старые ошибки;
- увеличивается стоимость и задержка;
- ответ становится менее устойчивым.

Исследователи и практики давно показывают: при длинном контексте модель не всегда одинаково хорошо держит всю информацию. Есть известный эффект, когда середина длинного контекста используется хуже, чем начало и конец. Поэтому задача не в том, чтобы “запихнуть всё”, а в том, чтобы *отобрать нужное*.

Здесь и появляется сам подход Context Engineering.

Не “впихнуть больше”. А:

- выбрать нужное;
- убрать лишнее;
- отделить важное от второстепенного;
- сжать без потери смысла;
- изолировать конфликтующие куски;
- обновлять контекст по ходу работы.

---

## 6. Что именно стало новой проблемой

Когда агент работает долго, его контекст начинает расти.

Он накапливает:

- сообщения;
- промежуточные выводы;
- результаты инструментов;
- куски документов;
- ошибки;
- исправления;
- напоминания;
- историю прошлых шагов.

И тут появляются новые вопросы:

- что оставить в контексте, а что выбросить;
- что записать в память, а что не тащить дальше;
- что искать через RAG, а что хранить как инструкцию;
- что надо сжать, а что нельзя терять;
- как не дать мусору отравить следующий шаг.

В современных агентных подходах часто выделяют несколько базовых действий с контекстом:

- `write` (записать);
- `select` (выбрать);
- `compress` (сжать);
- `isolate` (изолировать).

Это не просто красивые слова. Это реальные способы не развалить агента на длинной задаче.

---

## 7. Как тут появляется Context Engineering

Если очень просто, Context Engineering - это работа по сборке правильной рабочей среды для AI-системы.

В эту среду входят:

- задача;
- цель;
- роль;
- документы;
- источники истины;
- память;
- инструменты;
- правила;
- ограничения;
- формат результата;
- проверка;
- `stop conditions` (условия остановки - когда надо прекратить работу и спросить человека).

Поэтому Context Engineering отвечает уже не только за “как попросить”, а за гораздо более практичный вопрос:

Что модель должна видеть, знать, помнить, использовать и проверить, чтобы сделать задачу нормально?

Это и есть причина появления темы.

Не маркетинг. Не новое красивое слово. А смена устройства самой AI-работы.

---

## 8. Простая схема перехода

Вот базовая карта главы:

Chat -> Prompt -> Agent -> Context -> Verification

Разберём по шагам.

### Chat

Обычный разговор с моделью.

Здесь человек ждёт ответа.

### Prompt

Точнее сформулированный запрос.

Здесь человек уже понимает, что нужно задать роль, формат и границы.

### Agent

Система, которая умеет не только отвечать, но и выполнять действия.

### Context

Среда, в которой агент работает: задача, документы, память, инструменты, ограничения, правила.

### Verification

Проверка результата.

Без неё агент может “что-то сделать”, но не обязательно сделать правильно.

Эта схема нужна не ради красоты. Она помогает понять, в какой момент меняется сам принцип работы.

---

## 9. Где именно ломается старая логика

Старая логика ломается в пяти местах.

### 1. Запрос есть, а данных нет

Агенту говорят: “Сделай отчёт”. Но не говорят, где данные и какие считать правильными.

### 2. Запрос есть, а инструментов нет

Агенту нужно посмотреть CRM, но у него нет доступа или нет разрешения.

### 3. Запрос есть, а контекст противоречив

В одном месте написано одно правило, в другом - противоположное.



#### 4. Запрос есть, а памяти нет

Агент каждый раз начинает с нуля и не помнит важное.

#### 5. Запрос есть, а проверки нет

Агент что-то сделал, но никто не проверил, что именно.

Именно поэтому Context Engineering - это не про “умный промпт”. Это про устранение системных провалов.

---

## 10. Пример из жизни: отчёт по клиентам

### Плохой вариант

Сделай отчёт по клиентам.

Что пойдёт не так:

- непонятно, какие клиенты;
- непонятно, за какой период;
- непонятно, где брать данные;
- непонятно, можно ли открывать CRM;
- непонятно, что считать риском;
- непонятно, как проверить результат.

### Хороший вариант

Подготовь черновик отчёта по клиентам за последние 30 дней.

Цель: найти клиентов с риском оттока.

Источники:

- CRM;
- таблица оплат;
- последние заметки менеджеров.

Ограничения:

- не отправляй ничего клиентам;
- не меняй данные в CRM;
- если доступа не хватает - остановись и скажи, какой доступ нужен.

Формат результата:

- краткий вывод;
- таблица 5 клиентов с самым высоким риском;
- причина риска по каждому клиенту;
- список использованных источников.

Критерий готовности:

отчёт можно показать руководителю без дополнительного объяснения, откуда взялись данные.

Что здесь произошло?

Мы не просто улучшили формулировку. Мы собрали *контекст*.

А это уже работа контекстного инженера.

---

## 11. Почему это важно именно сейчас

Потому что AI перестал быть только “ответчиком”. Он становится частью реальной работы.

Сейчас AI всё чаще:

- читает документы;
- ищет по базе знаний;
- вызывает функции;
- работает в цепочке шагов;
- делает черновики;
- передаёт результат другому агенту или человеку;
- опирается на память и внешние данные;
- требует проверки перед действием.

А значит, ценность переносится с одного правильного запроса на целую архитектуру контекста.

И вот почему Context Engineering появился именно сейчас: потому что изменилась сама форма AI-работы.

---

## 12. Частые ошибки новичка

### Ошибка 1. Считать, что это просто длинный промпт

Нет. Это система: что дать, откуда взять, что убрать, что сохранить, что проверить.

### Ошибка 2. Пихать в контекст всё подряд

Лишнее вредит не меньше, чем нехватка данных.

### Ошибка 3. Не различать prompt и context

Prompt - просьба. Context - рабочая среда.

### Ошибка 4. Не задавать критерий готовности

Если не понять, как выглядит “готово”, агент может принести что угодно.

### Ошибка 5. Думать, что проблема всегда в модели

Часто проблема в контексте, источниках, правах, памяти или проверке.

---

## 13. Что забрать с собой из главы

Запомните четыре вещи:

1. Prompt engineering полезен, но это только первый слой.
2. Context Engineering нужен там, где AI-агент должен работать, а не просто отвечать.
3. Плохой контекст ломает даже сильную модель.
4. Правильная работа - это не “больше текста”, а “лучше собранная среда”.

Короткая формула главы:

Prompt = как попросить.  
Context = что дать для работы.  
Verification = как понять, что получилось.

## 14. Практика

### Задание

Возьмите любой свой реальный процесс, где вы уже пробовали использовать AI. Это может быть:

- отчёт;
- письмо;
- анализ;
- подборка;
- таблица;
- черновик документа;
- ответ клиенту;
- поиск по базе знаний.

Заполните карту:

Мой процесс:

Chat:

Что я обычно просто спрашиваю у AI?

Prompt:

Какой запрос я бы дал в лоб?

Agent:

Если бы это делал агент, какую роль он бы играл?

Context:

Какие данные, источники, правила, ограничения и память ему нужны?

Verification:

Как я пойму, что результат нормальный?

Что ломается без контекста:

Где агент начнёт гадать или ошибаться?

Что я добавлю в Context Engineering Pack:

Какой артефакт я должен собрать, чтобы следующая попытка была лучше?

### Ожидаемый результат

После заполнения у вас должен появиться не “просто промпт”, а маленькая карта рабочей среды.

Если у вас получается только одна строка вроде “надо лучше сформулировать запрос”, значит контекст ещё не собран.

## 15. Самопроверка

Ответьте себе честно:

- Я могу объяснить разницу между prompt и context без сложных слов?
- Я понимаю, почему агенту нужен не только запрос, но и рабочая среда?
- Я вижу, где у меня в реальной задаче не хватает данных, источников, прав или проверки?
- Я могу показать один плохой и один хороший пример контекста?
- Я понимаю, почему “больше контекста” не всегда лучше?

Если на все вопросы ответ “да” - глава усвоена. Если где-то “не очень” - перечитайте примеры и заполните карту ещё раз.

---

## 16. 60-секундное объяснение

Можно сказать так:

Context Engineering появился потому, что AI перестал быть просто чатом. Когда модель только отвечает, достаточно хорошего запроса. Но когда AI-агент должен делать работу, ему нужен не только prompt, а целая рабочая среда: задача, источники, память, инструменты, ограничения и проверка. Если контекст собран плохо, даже сильная модель начинает ошибаться. Поэтому новая задача - не просто писать запросы, а проектировать контекст так, чтобы агент мог реально и безопасно выполнить работу.

---

## 17. Что добавляем в Context Engineering Pack

Из этой главы в пакет идёт первая базовая карточка:

- Transition Map: Chat -> Prompt -> Agent -> Context -> Verification

Её можно потом использовать как стартовую схему для всей серии.

---

## 18. Короткий вывод

Context Engineering появился не ради красивого названия.

Он появился потому, что AI-системы изменились:

- они стали многократными, а не одноразовыми;
- они стали использовать инструменты;
- они стали опираться на память и источники;
- они стали делать реальные задачи в рабочем процессе;
- им понадобились правила, ограничения и проверки.

Если prompt engineering учил вас *просить*, то Context Engineering учит *собирать рабочую среду*.

И это уже другая дисциплина.

---

## 19. Источники и опорные материалы

Для этой главы использованы сильные опорные материалы и публичные сигналы:

- Anthropic - Effective context engineering for AI agents;
- Lance Martin / LangChain - Context Engineering for Agents и схема write / select / compress / isolate;
- Phil Schmid - обзор Context Engineering и перехода от prompt к context;
- HumanLayer / 12-factor-agents - идея own your context window;
- OpenAI Agents SDK - инструменты, guardrails (защитные ограничения), handoffs (передачи задачи) и context management (управление контекстом);
- Microsoft AI Agents for Beginners - образовательная линия по агентам;
- публичные упоминания термина у Tobi Lütke и Andrej Karpathy в обсуждениях и пересказах рынка.

### Как читать источники

Эти материалы нужны не для украшения списка, а как опора: они показывают, что разговор о контексте возник из практики построения AI-агентов, а не из желания придумать новое модное название.

В следующих главах источники будут использоваться точнее: отдельно для устройства LLM, отдельно для AI-агентов, отдельно для RAG, memory, tools, evals и безопасности.

## Полезные ссылки

### YouTube

Видео и разборы Алексея Ульянова

<https://youtube.com/@alekseiulianov>

### Telegram-канал Sprut AI

Публичные материалы, новости и разборы

[https://t.me/Sprut\\_AI](https://t.me/Sprut_AI)

### Чат Telegram-канала Sprut AI

Обсуждения и вопросы сообщества

<https://t.me/+eH-qNIDmud8zNDZi>

### AI Операционка

Закрытый продуктовый контур и материалы

<https://t.me/tribute/app?startapp=sJyg>